
A Small-Scale Experimental Study of the Muon Optimizer and Newton–Schulz Iterations

Zheng Renjie¹ Chen Ximing¹ Jiang Hongjun¹

Abstract

This project studies whether the Muon optimizer can provide faster and more stable training than AdamW on small neural networks, and how its behavior depends on the Newton–Schulz approximation used to compute matrix-orthogonalized updates. We implement a PyTorch training framework with AdamW and Muon, evaluate MLP and small CNN models on MNIST and Fashion-MNIST, run equal-budget hyperparameter search, and ablate the Newton–Schulz iteration count and coefficients. On Fashion-MNIST with an MLP, Muon improves final validation accuracy from $88.75\% \pm 0.49\%$ to $89.95\% \pm 0.28\%$ over five seeds, while reducing final training loss from 0.2063 to 0.0306. With a small CNN, Muon improves accuracy from $86.57\% \pm 0.71\%$ to $90.78\% \pm 0.65\%$. Newton–Schulz ablations show that the practical Muon coefficients converge faster than a standard polynomial choice, but large iteration counts can over-optimize the training loss without improving final validation accuracy. These results suggest that Muon is a fast and useful optimizer for matrix-valued hidden layers, but its benefits depend strongly on the iteration design and early-stopping behavior.

1. Introduction

Optimization is a central problem in modern deep learning. Adam and AdamW are widely used because adaptive coordinate-wise scaling gives stable performance across many tasks (Kingma & Ba, 2015; Loshchilov & Hutter, 2019). Muon is a more recent optimizer designed for hidden layers of neural networks (Jordan, 2024). Instead of applying a coordinate-wise adaptive update, Muon treats hidden-layer weights as matrices, applies momentum, and then approximately orthogonalizes the momentum matrix before updating the parameters.

The mathematical core of Muon is connected to polar decomposition and Newton–Schulz iterations (Higham, 2008; Kim & Oh, 2026). This makes it a useful topic for a mathematical foundations project: the optimizer is not only a

practical training method, but also an example of how numerical linear algebra changes optimization dynamics.

This report asks three questions:

1. Can Muon converge faster or achieve better validation accuracy than AdamW on small neural networks?
2. How sensitive is Muon to the Newton–Schulz iteration count q ?
3. Do the practical Muon coefficients behave differently from a standard Newton–Schulz polynomial choice?

The experiments intentionally use small and reproducible image-classification tasks rather than large language model training. This keeps the project computationally controlled while still exposing the main optimizer behavior.

1.1. Problem Formulation

All experiments solve empirical risk minimization for supervised classification:

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i), \quad (1)$$

where f_{θ} is an MLP or CNN, ℓ is cross-entropy loss, and θ contains matrix-valued hidden parameters as well as non-matrix parameters such as classifier-head weights. The project compares two update rules for minimizing Eq. (1): AdamW, which rescales coordinates using first- and second-moment estimates, and Muon, which applies a matrix-valued polar-direction update to hidden layers. The key experimental question is whether this matrix update improves optimization speed, validation accuracy, or stability under the same training budget.

2. Background

2.1. AdamW

AdamW maintains exponential moving averages of the stochastic gradient and squared gradient. Given gradient g_t ,

it updates

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (2)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t \odot g_t. \quad (3)$$

After bias correction, AdamW applies a coordinate-wise update and decoupled weight decay (Loshchilov & Hutter, 2019). This makes it a strong baseline because it is robust, easy to tune, and commonly used in neural network training.

2.2. Muon

Muon uses a matrix-geometric update for hidden-layer weights. For a matrix parameter W_t with gradient G_t , Muon first computes a momentum matrix

$$M_t = \beta M_{t-1} + (1 - \beta) G_t. \quad (4)$$

Ideally, the update direction is the polar factor

$$\text{Polar}(M_t) = U_t V_t^\top, \quad (5)$$

where $M_t = U_t \Sigma_t V_t^\top$ is an SVD. Computing an SVD at every optimizer step would be expensive, so practical Muon approximates this polar direction with a few Newton–Schulz iterations.

In this project, the Newton–Schulz update has the form

$$X_{j+1} = aX_j + (bX_jX_j^\top + c(X_jX_j^\top)^2)X_j, \quad (6)$$

for $j = 0, \dots, q - 1$. We compare the practical Muon coefficients

$$(a, b, c) = (3.4445, -4.7750, 2.0315)$$

with a more standard polynomial choice

$$(a, b, c) = \left(\frac{15}{8}, -\frac{5}{4}, \frac{3}{8} \right).$$

3. Experimental Setup

3.1. Tasks and Models

We evaluate supervised image classification on MNIST (LeCun et al., 1998) and Fashion-MNIST (Xiao et al., 2017). The main controlled setting is an MLP with two hidden linear layers. We also run a small CNN robustness check on Fashion-MNIST.

The MLP uses bias-free matrix layers so that optimizer behavior is easier to interpret. The small CNN uses two convolutional blocks followed by an adaptive pooling layer and a linear classifier. All models use cross-entropy loss.

Algorithm 1 Muon optimizer step

Require: Hidden matrix parameter W , gradient G , momentum M , learning rate α , momentum coefficient β , Newton–Schulz steps q , coefficients (a, b, c)

- 1: $M \leftarrow \beta M + (1 - \beta)G$
- 2: Reshape M to a matrix $\tilde{M} \in \mathbb{R}^{d_1 \times d_2}$ if W is convolutional
- 3: Normalize $X_0 \leftarrow \tilde{M} / (\|\tilde{M}\|_F + \epsilon)$
- 4: **for** $j = 0$ to $q - 1$ **do**
- 5: $X_{j+1} \leftarrow aX_j + (bX_jX_j^\top + c(X_jX_j^\top)^2)X_j$
- 6: **end for**
- 7: Reshape X_q back to the shape of W
- 8: $W \leftarrow (1 - \alpha\lambda)W - \alpha X_q$
- 9: Update non-matrix parameters separately with AdamW

3.2. Optimizer Protocol

AdamW is applied to all trainable parameters. Muon is applied only to matrix-valued hidden parameters: hidden linear weights and reshaped convolutional weights. Biases, normalization-like parameters, and classifier-head parameters are excluded from Muon and handled by AdamW when present. This follows the intended use of Muon as an optimizer for hidden layers rather than scalar or head parameters.

For the main MLP experiments, AdamW uses learning rate 10^{-3} and weight decay 10^{-4} . Muon uses learning rate 10^{-2} , weight decay 10^{-4} , $q = 3$, and the practical Muon coefficients. We also run an equal-budget search on Fashion-MNIST with 12 AdamW configurations and 12 Muon configurations. The search supports these representative choices: the best AdamW configuration is $(\alpha, \lambda) = (3 \cdot 10^{-3}, 10^{-4})$, while the best Muon configuration is $(10^{-2}, 10^{-4})$.

3.3. Metrics

We record training loss, validation loss, validation accuracy, epoch time, and peak GPU memory. Main comparisons are averaged over multiple random seeds. MNIST uses three seeds. Fashion-MNIST MLP and small CNN experiments use five seeds. Hyperparameter search uses one seed and is used only to verify the relative tuning budget, not as a final statistical estimate.

3.4. Reproducibility Details

The implementation is written in PyTorch (Paszke et al., 2019). Every run writes a CSV row for each epoch with dataset, model, optimizer, learning rate, weight decay, seed, training loss, validation loss, validation accuracy, epoch time, peak GPU memory, Newton–Schulz steps, and coefficient set. All figures and tables are generated from these

Table 1: MNIST sanity check.

Method	Acc.	Val Loss	Train Loss	Time
AdamW	98.05 $\pm$ 0.17	0.0706	0.0159	2.566
Muon $q = 3$	98.39 $\pm$ 0.09	0.0728	0.0025	3.264

Table 2: Fashion-MNIST MLP main comparison.

Method	Acc.	Val Loss	Train Loss	Time
AdamW	88.75 $\pm$ 0.49	0.3329	0.2063	2.558
Muon $q = 3$	89.95 $\pm$ 0.28	0.4424	0.0306	2.784

logs. Before the full experiments, I verified that AdamW and Muon each complete a one-epoch smoke test, that fixed seeds reproduce the same setup, and that Muon receives only matrix-valued hidden parameters. Experiments were run on an NVIDIA RTX PRO 6000 Blackwell Server Edition GPU with batch size 512. MNIST uses 10 epochs, Fashion-MNIST MLP uses 15 epochs, the small CNN main run uses 12 epochs, and small CNN ablations use 10 epochs.

4. Results

4.1. MNIST Sanity Check

MNIST is an easy benchmark, so it is mainly used to verify that both optimizers run correctly. Table 1 shows that AdamW and Muon both reach high validation accuracy. Muon has slightly higher final accuracy and much lower final training loss, but it is slower per epoch.

4.2. Fashion-MNIST MLP

Fashion-MNIST is the main experiment because it is harder than MNIST but still small enough for repeated runs. Table 2 and Figure 1 show that Muon improves final validation accuracy from 88.75% to 89.95% over five seeds. The final training loss is also much lower for Muon, decreasing from 0.2063 to 0.0306.

The validation loss tells a more nuanced story. Table 3 separates final-epoch metrics from best-epoch behavior. Muon reaches its best validation loss around epoch 5.2, while AdamW reaches it around epoch 13.0. Thus, Muon optimizes faster, but continuing training can overfit. This is also visible in the time-to-threshold metric: Muon reaches the selected training-loss threshold around epochs 8–9, while AdamW does not reach it within 15 epochs.

4.3. Hyperparameter Search

Table 4 reports the top configurations from the equal-budget search. The search is not intended as a final multi-seed comparison, but it reduces the risk that the main result comes

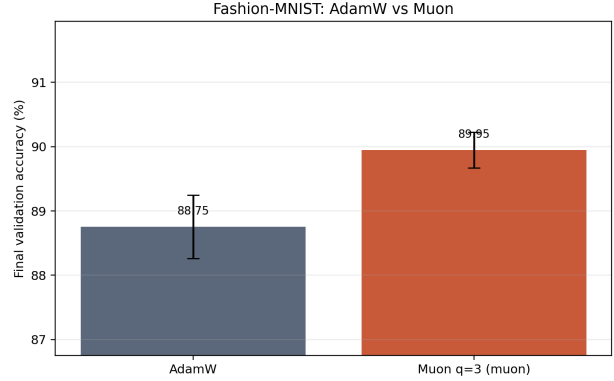


Figure 1: Fashion-MNIST MLP final accuracy.

Table 3: Fashion-MNIST MLP best-epoch metrics.

Metric	AdamW	Muon $q = 3$
Best val. acc.	89.09	90.25
Best acc. epoch	13.4	12.8
Best val. loss	0.3187	0.2970
Best loss epoch	13.0	5.2

from an unfair choice of learning rates. The best AdamW setting reaches 88.02% after eight epochs, while the best Muon setting reaches 90.25%.

4.4. Newton–Schulz Ablation on MLP

The Newton–Schulz ablation is the main mathematical experiment. Table 5 compares $q \in \{1, 2, 3, 5\}$ and two coefficient choices. With the practical Muon coefficients, increasing q sharply reduces training loss: from 0.2150 at $q = 1$ to 0.0062 at $q = 5$. However, final validation accuracy does not improve beyond $q = 3$, and final validation loss becomes much worse for $q = 5$.

The standard coefficients are more conservative. Their training loss decreases more slowly, and validation loss remains more stable. This supports the interpretation that the practical coefficients are not merely approximating the polar factor more accurately; they change the optimization dynamics by producing stronger updates.

4.5. Small CNN Robustness Check

To check whether the MLP result is architecture-specific, we repeat the AdamW vs Muon comparison with a small CNN. Table 6 and Figure 5 show a stronger result: Muon improves validation accuracy from 86.57% to 90.78%. In this setting, Muon also improves validation loss, unlike the final-epoch MLP result.

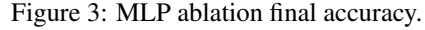
Method	LR	WD	Acc.	Val Loss
AdamW	0.003	0.0001	88.02	0.3452
AdamW	0.003	0.0100	87.69	0.3513
AdamW	0.003	0.0000	87.59	0.3583
Muon $q = 3$	0.010	0.0001	90.25	0.3033
Muon $q = 3$	0.010	0.0100	90.24	0.2997
Muon $q = 3$	0.010	0.0000	90.14	0.3045



5. Discussion

First, Muon consistently lowers training loss faster than AdamW. This is most visible on Fashion-MNIST MLP, where Muon reaches the selected training-loss threshold around epochs 8–9, while AdamW does not reach it within 15 epochs.

Third, Newton–Schulz coefficients matter. Practical Muon coefficients reach high validation accuracy with fewer iterations, while standard coefficients are more conservative and often need larger q . Therefore, the coefficient choice is part of the optimizer design, not only a numerical approximation



Fourth, the small CNN experiment shows that the result is not limited to MLPs. In fact, Muon gives a larger gain on the CNN in these runs. This may be because convolutional kernels are natural matrix-valued hidden parameters after reshaping, making them suitable for Muon’s matrix update. The CNN ablation is also a mixed result: unlike the MLP, increasing q to 5 remains competitive for the standard coefficients, suggesting that the best Newton–Schulz design is architecture-dependent.

This project is intentionally small-scale. The results do not prove that Muon is always better than AdamW, nor do they directly transfer to large language model training. The hyperparameter search uses one seed, so it should be treated as a fairness check rather than a final benchmark. The experiments also use fixed training budgets; some Muon configurations, especially large q , may look better under early stopping. Finally, the implementation uses a practical subset of Muon behavior and does not analyze all possible parameter-group protocols.

Table 5: Fashion-MNIST MLP Newton–Schulz ablation.

Coeff.	q	Acc.	Val Loss	Train Loss	Time
Muon	1	88.84 ± 0.13	0.3213	0.2150	2.779
Muon	2	89.67 ± 0.21	0.3392	0.1093	2.817
Muon	3	89.95 ± 0.28	0.4424	0.0306	2.843
Muon	5	89.85 ± 0.10	0.5652	0.0062	3.162
Standard	1	88.15 ± 0.20	0.3317	0.2556	2.882
Standard	2	88.86 ± 0.21	0.3203	0.2118	2.739
Standard	3	89.33 ± 0.16	0.3196	0.1625	2.888
Standard	5	89.80 ± 0.20	0.3814	0.0661	3.047

Table 6: Small CNN main comparison.

Method	Acc.	Val Loss	Train Loss	Time
AdamW	86.57 ± 0.71	0.3821	0.3497	2.953
Muon $q = 3$	90.78 ± 0.65	0.2677	0.1831	3.240

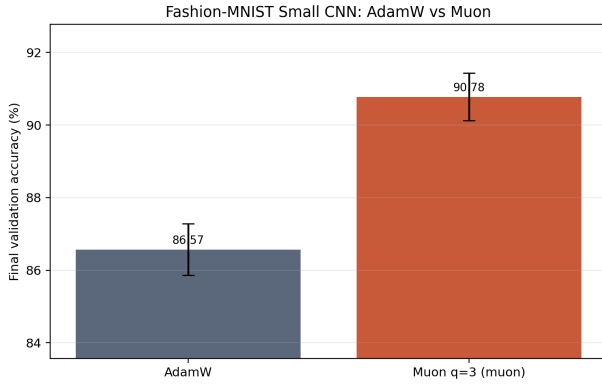


Figure 5: Small CNN final accuracy.

7. Conclusion

This project implemented a reproducible PyTorch study of AdamW, Muon, and Newton–Schulz iterations on small neural networks. Muon achieved faster loss reduction and higher validation accuracy than AdamW on Fashion-MNIST MLP and small CNN models. The Newton–Schulz ablations show that q and coefficient choice strongly affect the trade-off among convergence speed, validation accuracy, and computational cost. In the tested settings, $q = 3$ with practical Muon coefficients is a strong default: it is faster and more accurate than small q , while avoiding some of the overfitting behavior seen at $q = 5$. Overall, Muon is a promising optimizer for matrix-valued hidden layers, but its practical success depends on careful iteration design and evaluation beyond final training loss.

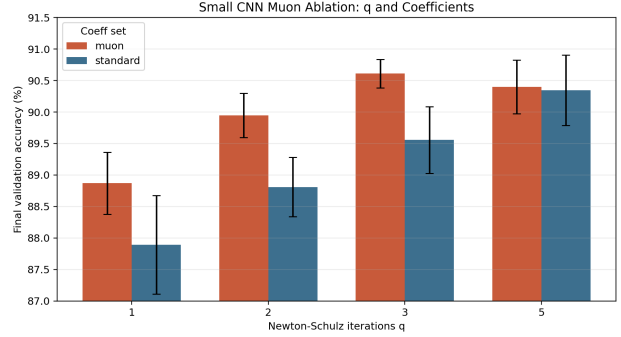


Figure 6: Small CNN ablation accuracy.

References

- Higham, N. J. *Functions of Matrices: Theory and Computation*. SIAM, 2008.
- Jordan, K. Muon: An optimizer for hidden layers in neural networks. <https://kellerjordan.github.io/posts/muon/>, 2024.
- Kim, G. and Oh, M. Convergence of muon with newton–schulz. In *International Conference on Learning Representations*, 2026.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *International Conference on Learning Representations*, 2015.
- LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. *International Conference on Learning Representations*, 2019.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. Pytorch: An imperative style, high-performance

Table 7: Small CNN ablation table.

Coeff.	q	Acc.	Val Loss	Train Loss	Time
Muon	1	88.87 ± 0.49	0.3138	0.2796	3.342
Muon	2	89.95 ± 0.35	0.2873	0.2312	3.254
Muon	3	90.61 ± 0.23	0.2683	0.2004	3.325
Muon	5	90.40 ± 0.42	0.2751	0.1963	3.385
Standard	1	87.89 ± 0.78	0.3435	0.3121	3.257
Standard	2	88.81 ± 0.47	0.3168	0.2825	3.436
Standard	3	89.56 ± 0.53	0.2986	0.2542	3.479
Standard	5	90.34 ± 0.56	0.2776	0.2160	3.458

deep learning library. In *Advances in Neural Information Processing Systems*, 2019.

Xiao, H., Rasul, K., and Vollgraf, R. Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.